

# SDLC Mapping: OrangeHRM Test Automation Project

---

**Project:** OrangeHRM Demo Automation (Login, Admin, PIM, Leave, Dashboard)

**AUT:** opensource-demo.orangehrmlive.com    **Author:** ABHISHEK K M    **Status:** Baseline v1.0

This project is a QA-side engagement against an already-built application, so the SDLC mapping below describes how the *automation project itself* moves through each phase, and how it hooks into the phases of the product it tests.

---

## Phase 1 — Requirements

**Generic purpose:** Capture what the system must do.

**For this project:**

- Treat the OrangeHRM demo application's existing behaviour as the requirements baseline (there is no formal SRS — behaviour is discovered by exploration, which mirrors many real brownfield engagements).
- Define automation requirements: which modules are in scope (Login, Admin user management, PIM employee records, Leave, Dashboard), which flows are business-critical, and which are explicitly out of scope (e.g., Buzz, Performance, mobile).
- Capture non-functional requirements for the framework itself: must run in both Cypress and Playwright (JavaScript), must run headless in CI, must produce shareable HTML reports, must tolerate the shared/reset nature of the public demo environment.

**Output:** scoped requirement list per module, automation feasibility notes (e.g., demo data resets periodically, so tests must create their own data).

---

## Phase 2 — Design

**Generic purpose:** Decide how the system will be built.

**For this project:**

- Produce the HLD (framework architecture, Page Object Model, tooling, CI) and LLDs (per-module page object and test design) — Artifacts 4 and 5.
- Design the locator strategy up front, because OrangeHRM's DOM (oxd-\* component classes, few stable IDs) drives most flakiness risk.
- Design test data management: fixture files for static data, runtime generation (unique employee names/IDs) for anything the test creates, since the demo site is shared.

**Output:** HLD document, LLD documents, folder-structure convention, naming conventions.

---

## Phase 3 — Build (Implementation)

**Generic purpose:** Write the code.

**For this project:**

- Implement page objects, fixtures, utilities, and spec files for each module — Login and Dashboard first (smallest, gate everything else), then Admin, PIM, Leave.
- Two parallel implementations (Cypress and Playwright) sharing the same design and test cases, which demonstrates tool-agnostic framework thinking.
- Code review conventions apply to test code exactly as they would to product code: PRs per module, lint (ESLint), no hard-coded waits, no credentials in code (env variables / .env excluded from git).

**Output:** working automation code per module, merged via reviewed PRs.

---

## Phase 4 — Test

**Generic purpose:** Verify the system.

**For this project (testing the tests):**

- Verify the automation itself: run each suite locally 3× consecutively to shake out flakiness before it is considered "done".
- Cross-browser verification (Chromium, Firefox, WebKit in Playwright; Chrome, Edge in Cypress).
- Negative verification: deliberately break a locator or assertion and confirm the failure report is diagnosable (screenshot, trace, clear message).

**Output:** stable suites with a documented flake rate (target: <2% over 10 CI runs).

---

## Phase 5 — Deploy

**Generic purpose:** Release to users.

**For this project:**

- "Deployment" = wiring suites into CI (GitHub Actions): smoke suite on every push/PR, full regression nightly against the demo URL.
- Publish HTML reports as CI artifacts; failures notify via the PR check status.
- Tag a versioned release of the framework once all five modules are automated.

**Output:** green CI pipeline, published reports, tagged release.

---

## Phase 6 — Maintain

**Generic purpose:** Keep the system healthy over time.

**For this project:**

- The OrangeHRM demo is periodically upgraded and its data resets — expect locator and data breakage. Triage failures weekly: product change vs. test defect vs. environment flake.
- Maintain a known-issues list for demo-environment quirks (e.g., slow cold starts, shared-tenant data collisions).
- Refactor page objects as duplication appears; keep both tool implementations in feature parity.

**Output:** changelog, updated suites, quarterly review of coverage vs. scope.

---

## Summary table

| SDLC PHASE   | THIS PROJECT'S ACTIVITY                                    | KEY OUTPUT                  |
|--------------|------------------------------------------------------------|-----------------------------|
| Requirements | Scope 5 modules, discover behaviour, define framework NFRs | Scoped requirements list    |
| Design       | HLD + LLD, locator & data strategy                         | Artifacts 4 & 5             |
| Build        | Page objects, specs, fixtures in Cypress + Playwright      | Reviewed, merged code       |
| Test         | Stabilise suites, cross-browser, failure diagnosability    | Flake rate < 2%             |
| Deploy       | GitHub Actions smoke + nightly regression, reports         | Green pipeline, release tag |
| Maintain     | Weekly triage, locator/data upkeep, parity between tools   | Changelog, coverage review  |